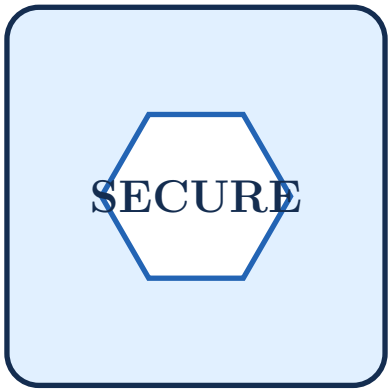
CyberShield OS

A Security-Focused Mini Operating System



Operating Systems Course Project

Project Objective

To develop a lightweight Linux-based security monitoring system that demonstrates operating-system concepts such as process management, resource management, file systems, user authentication, system logging, and security event detection.

Team Size: 5 Students
Estimated Development Time: 1 Month
Project Difficulty: Easy to Moderate

Department of Computer Science and Engineering

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Project Concept 3

3 Problem Statement **3**

4 Project Objectives **4**

5 Project Scope **4**
5.1 Included Features 4
5.2 Features Outside the Initial Scope 5

6 System Architecture **5**

7 Major Modules **6**
7.1 Process Monitoring Module 6
7.2 File Integrity Monitoring Module 6
7.3 Login Monitoring Module 7
7.4 Resource Monitoring Module 7
7.5 Security Alert Module 8

8 System Workflow **8**

9 Technology Stack **8**

10 Operating Systems Concepts Demonstrated **9**

11 Team Division **9**

12 One-Month Development Plan **9**
12.1 Development Timeline 10

13 Proposed User Interface **10**

14 Example System Output **11**

15 Proposed Project Structure **11**

16 Basic Detection Logic **12**
16.1 Process Detection 12
16.2 File Integrity Detection 12
16.3 Login Detection 12

17 Testing Strategy **12**

18 Expected Outcomes **13**

19 Future Enhancements **13**

20 Conclusion **14**

1 Abstract

CyberShield OS is a lightweight security-focused system designed as an Operating Systems course project. Instead of attempting to develop a complete operating system kernel from scratch, the project implements a security monitoring layer on top of a Linux-based operating system.

The system continuously observes important operating-system activities such as running processes, CPU and memory utilization, file modifications, login attempts, and system events. Suspicious activities are identified using simple predefined security rules and are reported through a centralized alert mechanism.

The project is intentionally designed for a five-member team and a development period of approximately one month. Each member can develop an independent module, after which all modules are integrated into a single security monitoring system.

The project demonstrates practical applications of Operating Systems concepts including process management, memory management, file systems, user management, system logs, resource monitoring, and system-level interaction.

2 Introduction

Modern computer systems continuously execute processes, access files, manage memory, communicate with users, and record system events. These activities are essential for normal operation, but unusual behavior may indicate a security problem.

CyberShield OS aims to provide a simple security monitoring layer that observes these activities and produces understandable alerts.

The project is not intended to replace a commercial antivirus, intrusion detection system, or complete operating system. Instead, it is an educational project that connects security concepts with fundamental Operating Systems concepts.

2.1 Project Concept

The basic concept can be represented as:

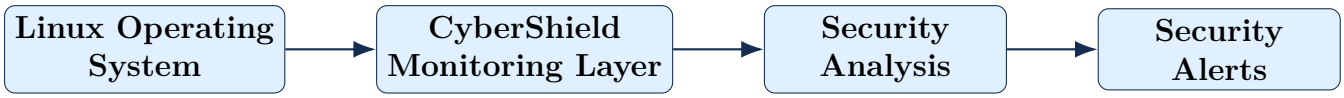


Figure 1: Basic CyberShield OS Concept

3 Problem Statement

Traditional Operating Systems provide mechanisms for process management, memory management, file management, and user authentication. However, users may not always have a simple interface for understanding suspicious system activity.

For educational purposes, there is a need for a small system that can demonstrate how Operating Systems information can be collected and analyzed for basic security monitoring.

CyberShield OS addresses this problem by creating a lightweight monitoring system capable of:

- Monitoring running processes.
- Monitoring CPU and memory usage.
- Detecting modifications to selected files.
- Monitoring login attempts.
- Recording security events.
- Generating alerts for suspicious activity.

4 Project Objectives

The major objectives of CyberShield OS are:

1. Demonstrate practical Operating Systems concepts.
2. Monitor important system resources.
3. Detect basic abnormal system behavior.
4. Monitor file integrity.
5. Track login and authentication events.
6. Generate centralized security alerts.
7. Provide an understandable security dashboard.
8. Develop the project within approximately one month.

5 Project Scope

The project is intentionally limited to features that can realistically be implemented by five students within one month.

5.1 Included Features

- Process monitoring.
- CPU monitoring.
- Memory monitoring.
- Disk monitoring.
- File integrity monitoring.
- Login attempt monitoring.

- Security event logging.
- Alert generation.
- Basic dashboard.

5.2 Features Outside the Initial Scope

The following features are intentionally excluded from the first version:

- Development of a complete operating-system kernel.
- Development of a custom bootloader.
- Full antivirus functionality.
- Advanced artificial-intelligence threat detection.
- Complete intrusion detection system.
- Advanced network firewall.
- Advanced malware reverse engineering.

This controlled scope makes the project achievable within the one-month development period.

6 System Architecture

The architecture of CyberShield OS consists of several independent monitoring modules connected to a central security engine.

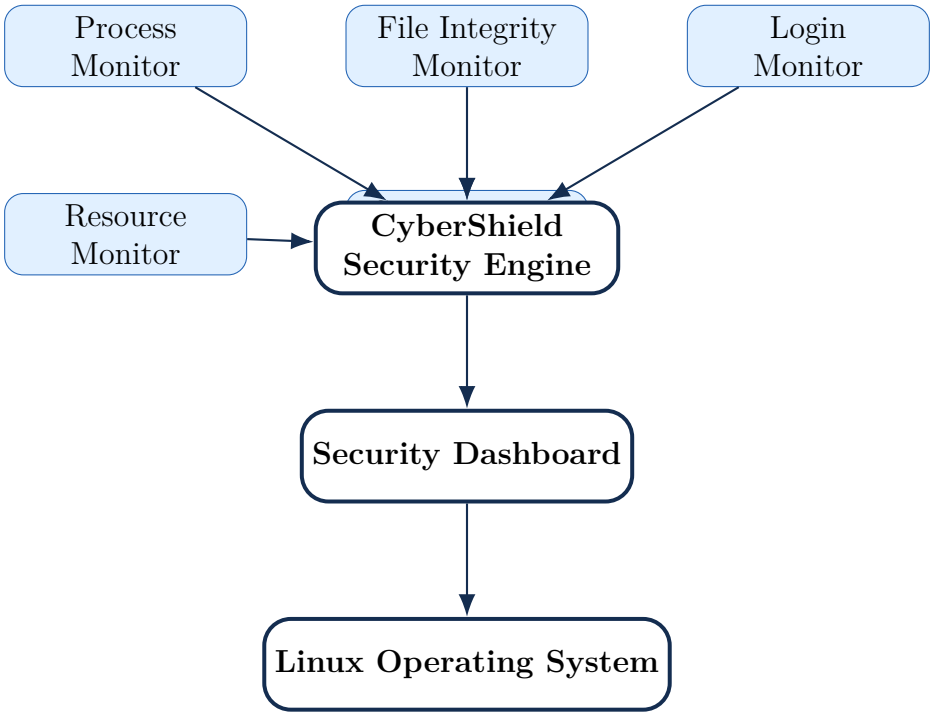


Figure 2: CyberShield OS System Architecture

7 Major Modules

7.1 Process Monitoring Module

The Process Monitoring Module observes processes currently executing on the Linux system.
Information collected may include:

- Process ID (PID).
- Process name.
- CPU utilization.
- Memory utilization.
- Process owner.
- Process status.

A simple rule can identify processes consuming unusually high resources.

PROCESS MONITOR
PID: 3021
Name: unknown
CPU: 87%
Memory: 9.2%
Status: **WARNING**

7.2 File Integrity Monitoring Module

The File Integrity Monitor checks whether selected files have been modified.
A cryptographic hash such as SHA-256 can be generated for a file. The original hash is stored and later compared with the current hash.

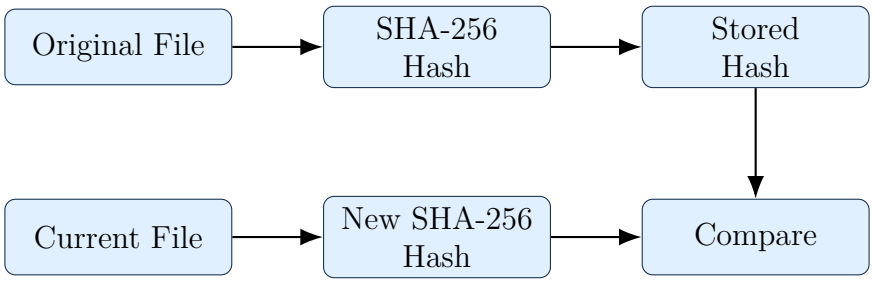


Figure 3: File Integrity Detection Process

If the two hashes differ, the system generates a file modification alert.

7.3 Login Monitoring Module

The Login Monitoring Module analyzes authentication information provided by the Linux operating system.
It can identify:

- Successful login attempts.
- Failed login attempts.
- Number of failed attempts.
- Time of authentication events.

A simple detection rule may be:

> If the number of failed login attempts exceeds a predefined threshold within a given time period, CyberShield generates a security warning.

7.4 Resource Monitoring Module

This module monitors:

- CPU usage.
- RAM usage.
- Disk utilization.
- Number of active processes.

A graphical representation can be displayed on the dashboard.

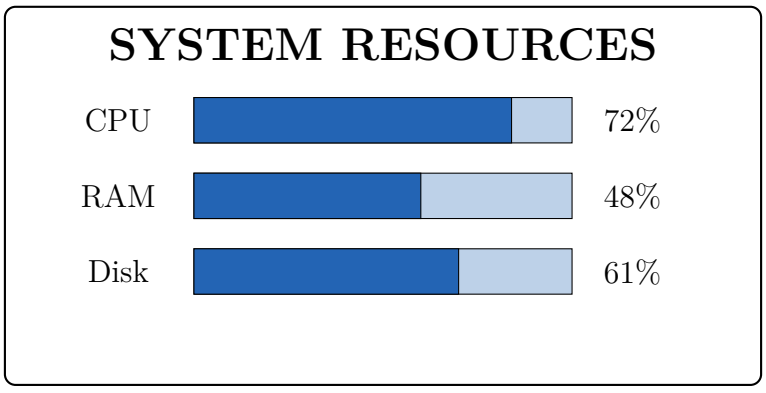


Figure 4: Example System Resource Dashboard

7.5 Security Alert Module

The Security Alert Module receives events from all monitoring modules and assigns a basic severity level.

HIGH: Multiple failed login attempts

MEDIUM: File modification detected

LOW: High resource utilization

8 System Workflow

The overall workflow of CyberShield OS is:

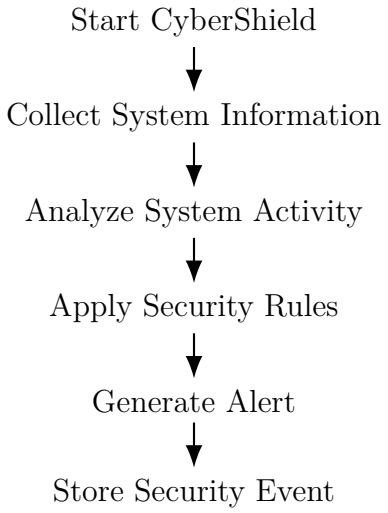


Figure 5: CyberShield OS Workflow

9 Technology Stack

Component	Technology
Operating System	Linux / Ubuntu / Kali Linux
Programming Language	Python
Process Monitoring	psutil
File Hashing	hashlib
File Operations	os / pathlib
System Commands	subprocess
Logging	Python logging module
Database	SQLite (optional)
GUI	Tkinter or lightweight web interface
Version Control	Git and GitHub

10 Operating Systems Concepts Demonstrated

CyberShield OS connects practical implementation with several Operating Systems concepts.

OS Concept	CyberShield Implementation
Process Management	Process Monitor
CPU Management	CPU Utilization Monitor
Memory Management	RAM Monitoring
File System	File Integrity Monitor
User Management	Login Monitoring
System Calls	Interaction with Linux system resources
Resource Management	CPU, RAM and disk monitoring
System Logging	Security Event Logger
File Permissions	File access and integrity checks
Inter-process/System Interaction	Security monitoring modules

11 Team Division

The project is divided among five students so that every member has a clearly identifiable contribution.

Member	**Module**	Responsibilities
1	**Process Monitor**	Process information, PID monitoring, CPU and process status
2	**File Integrity**	Hash generation, file monitoring, modification detection
3	**Login Monitor**	Authentication logs, failed-login detection, event analysis
4	**Resource Monitor**	CPU, RAM, disk monitoring and dashboard development
5	**Alert and Integration**	Alert engine, event logging, module integration and testing

12 One-Month Development Plan

The project is planned for approximately four weeks.

Week	Main Activities	Expected Result
Week 1	Linux environment setup, project structure, process monitoring, resource monitoring	Basic monitoring system
Week 2	File integrity monitoring and login monitoring	Core security modules completed
Week 3	Alert engine, logging, dashboard and integration	Integrated CyberShield system
Week 4	Testing, debugging, documentation and presentation	Final working project

12.1 Development Timeline

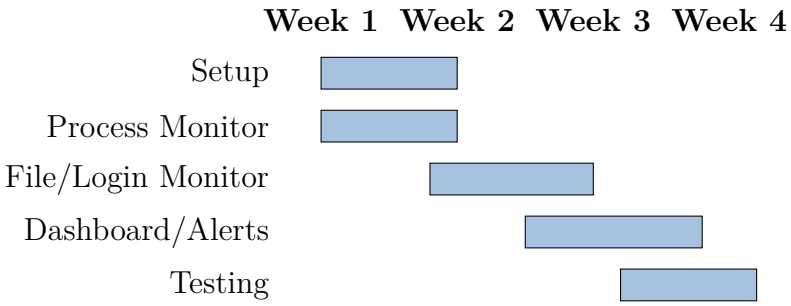


Figure 6: Approximate Four-Week Development Timeline

13 Proposed User Interface

A simple dashboard can provide the user with a centralized view of system health and security events.

Figure 7: Conceptual CyberShield Dashboard

14 Example System Output

A terminal-based version can initially produce output such as:

```
========================================
         CYBERSHIELD OS
    SECURITY MONITORING SYSTEM
========================================

SYSTEM STATUS
----------------------------------------
CPU Usage : 42%
Memory Usage : 51%
Disk Usage : 63%
Running Processes : 137

SECURITY STATUS
----------------------------------------
File Integrity : SAFE
Login Monitor : WARNING
Process Monitor : SAFE

RECENT ALERTS
----------------------------------------
[HIGH] Multiple failed login attempts
[MED] File modification detected
[LOW] High CPU process detected

========================================
SYSTEM STATUS: SECURITY WARNING
========================================
```

15 Proposed Project Structure

A simple modular structure can be used:

```
CyberShield-OS/
|
+-- main.py
|
+-- modules/
| +-- process_monitor.py
| +-- file_monitor.py
| +-- login_monitor.py
| +-- resource_monitor.py
|
+-- security/
| +-- alert_manager.py
| +-- event_logger.py
|
+-- dashboard/
```

```
| +-- dashboard.py
|
+-- database/
| +-- cyber_events.db
|
+-- config/
| +-- config.json
|
+-- tests/
| +-- test_process.py
| +-- test_file.py
| +-- test_login.py
|
+-- README.md
```

16 Basic Detection Logic

The initial version of CyberShield does not require complex artificial intelligence or machine learning. Simple rule-based detection is sufficient for the project.

16.1 Process Detection

For example:

$$CPU\ Usage > Threshold \Rightarrow Potentially\ Suspicious\ Process$$

16.2 File Integrity Detection

$$Hash_{original} \neq Hash_{current} \Rightarrow File\ Modification\ Detected$$

16.3 Login Detection

$$Failed\ Attempts > Threshold \Rightarrow Authentication\ Warning$$

These rules keep the implementation understandable and suitable for a one-month academic project.

17 Testing Strategy

Each module will be tested independently before integration.

Module	Test Case	Expected Result
Process Monitor	Start a CPU-intensive process	Process should appear in monitor
File Monitor	Modify monitored file	Modification alert generated
Login Monitor	Generate failed login attempts	Failed attempts detected
Resource Monitor	Increase system workload	CPU/RAM values update
Alert System	Trigger security condition	Alert generated and logged

18 Expected Outcomes

At the completion of the project, the team should have a working Linux-based security monitoring system capable of:

- Displaying system resource information.
- Listing running processes.
- Detecting selected file modifications.
- Monitoring authentication events.
- Generating security alerts.
- Maintaining security event logs.
- Providing a simple security dashboard.

The project should also provide students with practical experience in Linux, Python, system programming, Operating Systems, and basic cybersecurity.

19 Future Enhancements

If additional development time becomes available, the following features can be added:

- Network connection monitoring.
- Firewall integration.
- Email security notifications.
- More advanced anomaly detection.
- Web-based dashboard.
- Role-based user access.
- Database-based historical analysis.
- Real-time graphical charts.
- Optional machine-learning-based anomaly detection.

These features are intentionally treated as future enhancements rather than mandatory components.

20 Conclusion

CyberShield OS is a practical and manageable Operating Systems project that combines fundamental OS concepts with basic cybersecurity monitoring.
The project does not attempt to create a complete operating system kernel. Instead, it develops a security-focused monitoring layer that interacts with Linux system resources and demonstrates how operating-system information can be used for security analysis.
The modular architecture allows five students to work independently on different components while keeping the final system integrated. The proposed four-week development schedule makes the project realistic for a college semester project.
The final system will demonstrate process management, resource management, file-system interaction, user authentication, system logging, and security event detection through a single coherent application.

CyberShield OS

Monitor. Detect. Alert. Protect.